

Memory, Repulsion, and Attraction: Phase Transitions in Non-Markovian Random Walks on a Two-Dimensional Lattice

Max Zhang
NCMC, 2026
July 31, 2026

Abstract

In the problem of the classical random walk, the recurrence and transience of random walks are fundamentally determined by the spatial dimension, as established by Pólya’s celebrated theorem (1921): symmetric simple random walks are recurrent in one and two dimensions, yet transient in three and higher dimensions. However, this classical conclusion rests on the assumptions of memorylessness and isotropic stochasticity, which often fail in real-world scenarios where foraging animals avoiding recently visited locations and disoriented individuals in mist without map trying to find the exit. This raises a central question: what would happen if we model the true scenario of human explorer instead of a mathematically perfect condition, and what interesting conclusions may be discovered?

In this work, we propose a non-Markovian random walk model on a two-dimensional square lattice that incorporates three interacting mechanisms: an exponentially decaying memory field that allows the walker to remember the past visits, a repulsive bias that discourages the walker from approaching areas with high accumulated memory intensity within a local neighborhood, and an optional central attractive force that models the tendency to return to the origin. The competition between repulsion and attraction forces is defined by a Logistic soft threshold, clearly models the true human behavior.

We conduct extensive simulations on a 100×100 grid. For each parameter configuration, we repeat the simulation 100 times to obtain statistically robust estimates of their effect on escape time. We systematically scan the key parameters—the memory decay rate λ , the Logistic steepness k , and the repulsion threshold c —and perform nonlinear regression to fit the observed escape times against each parameter. We further investigate the influence of the central attraction strength η on the walker’s fate.

Keywords: random walk; exponential memory decay; Logistic repulsion; central attraction; phase transition; Monte Carlo simulation.

Introduction

The conclusion of classical random walks is that a drunkard wandering randomly on a two-dimensional plane will eventually return to every previously visited location infinitely many times. Yet, it rests on two critical assumptions that are rarely satisfied in real-world scenarios: (i) the walker possesses no memory of where it has been, and (ii) the walker has no preference or aversion toward any particular region of space.

The classic Self-Avoiding Walk (SAW) model captures this repulsive mechanism and is known to be transient in 2d, in stark contrast to the simple random walk. However, the SAW model assumes perfect memory of the walker, so the 2d plane will eventually be filled with past steps. These observations motivate a unifying question: what happens when a random walker is equipped with a decaying memory of its own trajectory and a preference to explore new places?

Our primary research questions are as follows:

1. How does the rate of memory decay affect the walker's efficiency in escaping a finite domain? Does slower decay always promote faster escape, or is there an optimal intermediate regime?
2. How sensitive is the walker's fate to the repulsion threshold parameter? Can a small change in threshold induce a dramatic shift in escape time?
3. When a central attractive force is introduced to model the 'return-to-safety' instinct, does it merely slow down the escape, or can it qualitatively reverse the walker's ultimate fate?
4. Finally, the question of transient or recurrence.

Theoretical background

1. Pólya's Recurrence Theorem

The starting point of any serious study of random walk recurrence is Pólya's celebrated theorem. Consider a simple symmetric random walk on the d -dimensional integer lattice $\mathbb{Z}^d$: at each step, the walker moves to one of the $2d$ nearest neighbors with equal probability $1/(2d)$. Pólya (1921) established the following theorem.

Theorem 1 (Pólya's Recurrence Theorem). A simple symmetric random walk on $\mathbb{Z}^d$ is recurrent for $d = 1, 2$ and transient for $d \geq 3$.

A random walk is said to be recurrent if it returns to its starting point with probability one; it is transient if there is a positive probability that it never returns. If we denote by p_{esc} the probability that the walker never returns to the origin, then recurrence means $p_{\text{esc}} = 0$, while transience means $p_{\text{esc}} > 0$.

In two dimensions, the return probability decays as $\mathbb{P}(S_{2n} = 0) \sim 1/(\pi n)$, and since $\sum_n 1/n$ diverges, the Borel-Cantelli argument yields recurrence. In three dimensions, however, $\mathbb{P}(S_{2n} = 0) \sim Cn^{-3/2}$, and $\sum_n n^{-3/2}$ converges, implying a positive escape probability.

2. The Self-Avoiding Walk

The SAW is one of the most important models in statistical mechanics. Unlike the simple random walk, the SAW is not a Markov process: it prohibits revisiting sites more than once, which makes the model difficult to analyze rigorously.

From the perspective of recurrence versus transience, the SAW in two dimensions is transient: there is a positive probability that the walker never returns to the origin, and in fact the walker eventually escapes to infinity almost surely.

3. The True Self-Avoiding Walk

A related but distinct model is the true self-avoiding walk (TSAW), introduced by Amit, Parisi, and Peliti in 1983. The TSAW 'tries to avoid places he has already visited', but unlike the standard SAW, the avoidance is probabilistic rather than absolute.

The TSAW occupies an intermediate position between the memoryless simple random walk and the strictly self-avoiding walk: it has infinite memory (the entire past influences future transitions), but the influence decays through the soft repulsion mechanism rather than through absolute prohibition, which is quite similar to the model that we will introduce in this paper.

4. Finite Memory and Non-Markovian Random Walks

A common feature of the SAW and TSAW is that they possess infinite memory: the walker's future behavior depends on the entire past trajectory. This makes them fundamentally non-Markovian.

Finite-memory random walks have been studied extensively in the probabilistic literature. A key result is that finite-memory perturbations of simple random walks often preserve recurrence in two dimensions. More generally, the recurrence or transience of finite-memory walks can be characterized in terms of the probabilities to persist in or switch direction. Intuitively, if the walker tends more to switch directions, then it will wander aimlessly; yet if it tends more, even slightly, to persist in one direction, then it is likely to escape.

Model definition

1. Space and Kinematics

We consider a finite two-dimensional square lattice

$$\Lambda = \{(i, j) \in \mathbb{Z}^2 : 0 \leq i, j \leq L - 1\},$$

where $L = 100$ is the linear size of the domain. The walker starts at the center of the lattice:

$$\mathbf{x}_0 = \left(\frac{L}{2}, \frac{L}{2}\right) = (50, 50).$$

Two sites $\mathbf{u} = (u_1, u_2)$ and $\mathbf{v} = (v_1, v_2)$ are adjacent if

$$|u_1 - v_1| + |u_2 - v_2| = 1.$$

Thus at each step the walker may move to one of the four cardinal neighbors, as in the classical simple random walk.

Unlike the classical random walk, however, our walker possesses an internal state encoding its current heading direction. We define the direction space

$$\mathcal{D} = \{(1, 0), (-1, 0), (0, 1), (0, -1)\},$$

corresponding to East, West, North, and South, respectively. The walker's state at time n is the pair

$$(\mathbf{x}_n, \mathbf{d}_n) \in \Lambda \times \mathcal{D},$$

where $\mathbf{x}_n$ is the current position and $\mathbf{d}_n$ is the current heading.

At each step, the walker may choose to move to any of the four neighboring sites. In terms of the heading direction $\mathbf{d}_n$, these four candidates are:

$$\begin{aligned} \text{Forward: } & \mathbf{d}_n, \\ \text{Backward: } & -\mathbf{d}_n, \\ \text{Left: } & \mathbf{d}_n^\perp = (-d_{n,2}, d_{n,1}), \\ \text{Right: } & -\mathbf{d}_n^\perp = (d_{n,2}, -d_{n,1}). \end{aligned} \tag{1}$$

All four directions are always permissible. Specifically, a walker that attempts to reverse direction will typically face a higher memory intensity, and will therefore be discouraged from doing so through the soft repulsive mechanism. However, the option remains available, ensuring that the walker retains full freedom to explore the space without restrictions.

The lattice boundary is treated as absorbing: if the walker attempts to move to a site outside Λ , the walk terminates and the escape time is recorded. We define the escape time as

$$\tau = \inf\{n \geq 1 : \mathbf{x}_n \notin \Lambda\}.$$

2. Exponentially Decaying Memory Strength Field

The first component of our model is the memory strength field, which records the walker's past visits with an exponential decay. Unlike hard cut-off memory windows, this formulation assigns non-zero weight to all past visits, with the influence of older visits decaying smoothly over time.

For each site $u \in \Lambda$, we denote by $t_j(u)$ the sequence of times at which the walker visited u . The memory strength of site u at the current time n is defined as the maximum of the exponentially decaying contributions from all past visits:

$$S_u(n) = \max_{j: t_j(u) \leq n} \{e^{-\lambda(n-t_j(u))}\}, \tag{2}$$

with the convention that $S_u(n) = 0$ if the site has never been visited. The parameter $\lambda > 0$ controls the rate of memory decay.

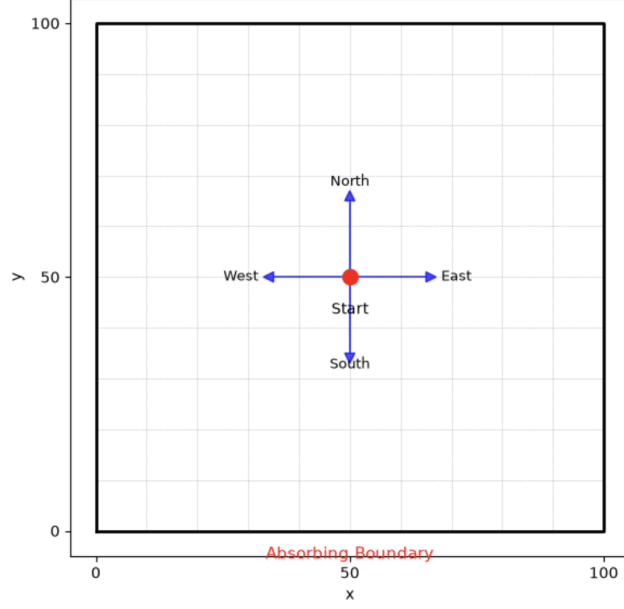


Figure 1: Schematic of the 100×100 lattice with absorbing boundary. The walker starts at the center

First, the use of the maximum rather than the sum ensures that repeated visits to the same site do not artificially inflate the memory intensity. In other words, a site visited many times long ago does not carry more memory weight than a site visited once very recently.

Second, the exponential decay is memoryless in the sense that the ratio of memory strengths at two sites depends only on the difference between their last visit times:

$$\frac{S_u(n)}{S_v(n)} = e^{-\lambda(t_{\text{last}}(u) - t_{\text{last}}(v))},$$

provided both sites have been visited.

The time required for a memory trace to decay to half of its initial strength is

$$t_{1/2} = \frac{\ln 2}{\lambda}.$$

We shall fix $\lambda = 0.04$ in our baseline simulations. This choice ensures that after 15 steps, the memory strength retains

$$e^{-0.04 \times 15} = e^{-0.6} \approx 0.549 > 0.5,$$

Figure 2 shows the memory decay curve with a 15 step cutoff. Any memory strength above 0.5 is considered to be significant, which contributes more to the calculation of memory intensity.

Now, for a candidate destination site $v \in \Lambda$ adjacent to the walker's current position x_n , we define the local memory intensity in the neighborhood of v as the sum of memory strengths over the 5×5 square centered at $\mathbf{v}$:

$$X_v(n) = \sum_{u \in N_{5 \times 5}(v)} S_u(n), \quad (3)$$

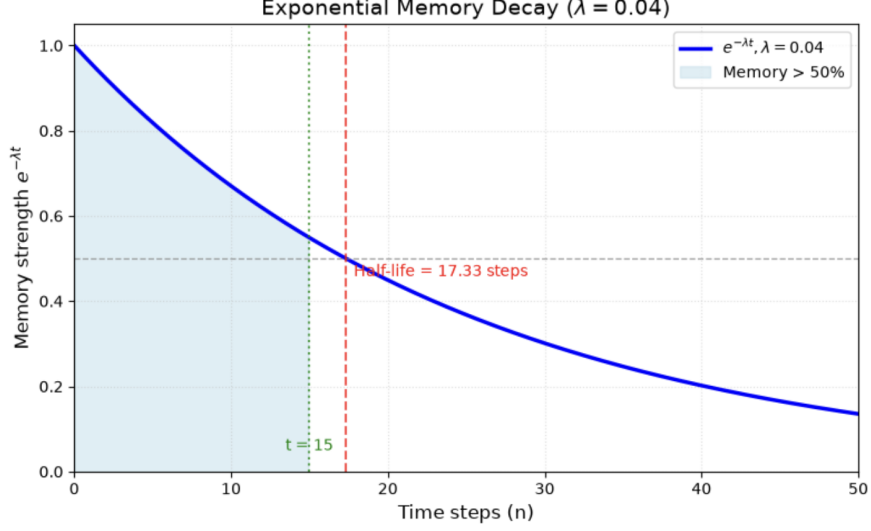


Figure 2: Exponential decay function $e^{-\lambda t}$ for $\lambda = 0.04$

where

$$N_{5 \times 5}(v) = \{v + (i, j) : i, j \in \{-2, -1, 0, 1, 2\}\} \cap \Lambda.$$

The 5×5 neighborhood includes the site v itself and the 24 surrounding sites. This spatial window means that the walker's perception is not limited to the immediate target site but extends to the surrounding local environment. It also cannot spread too far across the entire plane, which would result in the model too similar to TSAW.

A candidate site that is itself unvisited but is surrounded by recently visited sites should be repulsive, yet the probability should never be zero to prevent the condition of being 'trapped' by having no available paths to choose.

Figure 3 shows how memory intensity change over time when the walker moves to other places, and also shows the 5×5 boundary where probability is calculated for the next step.

Note that the memory field can be updated efficiently. At each step, after the walker moves to a new site x_{n+1} , we set

$$S_{x_{n+1}}(n+1) = 1$$

(the most recent visit), while for all other sites $u \neq x_{n+1}$, the memory strength decays multiplicatively:

$$S_u(n+1) = e^{-\lambda} S_u(n).$$

This update rule follows directly from the definition in Equation (3.1) and ensures that the memory field evolves without requiring the storage of the full visit history. The total memory intensity $X_v(n)$ can then be recomputed from the current memory field in $O(25)$ operations per candidate site, making the simulation computationally efficient.

3. Repulsive Probability

The second component of our model is the repulsive weight function, which translates the local memory intensity $X_v(n)$ into a probability bias against moving to candidate site v . We adopt a soft constraint based on the Logistic function.

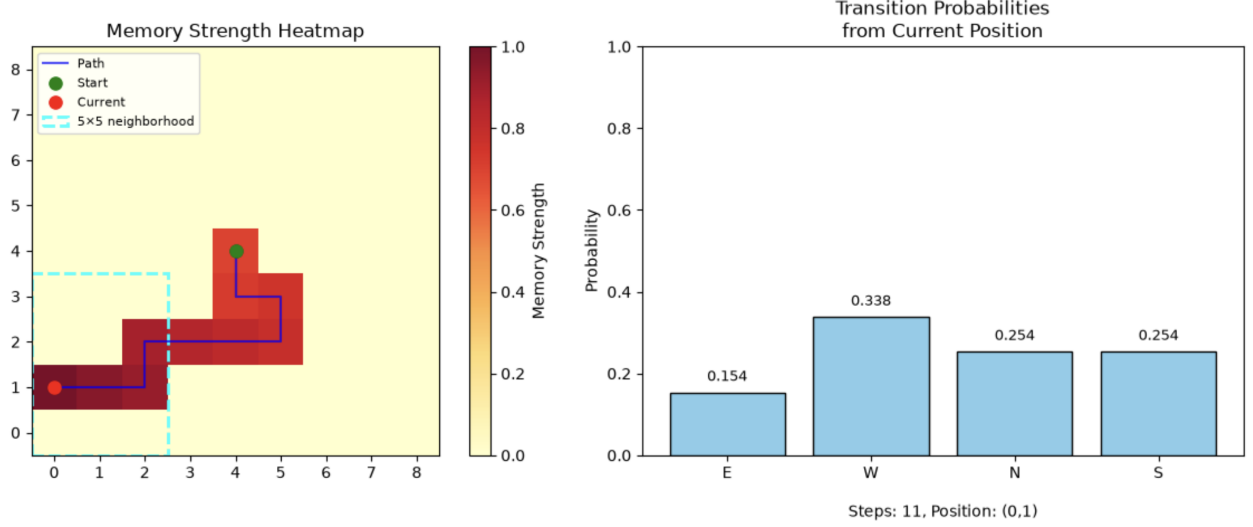


Figure 3: Illustration of the 5×5 neighborhood centered at the current walker position on a 9×9 lattice with heat map of past steps before exiting and sample probability of going to four directions

The Logistic function, also known as the sigmoid function, is defined as

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

For our purposes, we define the repulsive weight for a candidate site v as

$$W_{\text{rep}}(v; n) = \frac{1}{1 + e^{k(X_v(n) - c)}}, \quad (4)$$

where:

- $X_v(n)$ is the total memory intensity within the 5×5 neighborhood of v , defined in Equation (3.2);
- $k > 0$ is the steepness parameter that controls the sharpness of the transition;
- $c > 0$ is the repulsion threshold that determines the midpoint of the transition.

The Logistic function is particularly well-suited for modeling soft thresholding behavior. To understand its properties, consider the following limiting cases:

1. When $X_v(n) \ll c$ (i.e., the local memory intensity is well below the threshold), the exponent $k(X_v(n) - c)$ is a large negative number, so $e^{k(X_v(n) - c)} \approx 0$, and

$$W_{\text{rep}}(v; n) \approx 1.$$

The candidate site is essentially unrepelled, and the walker is free to move there.

2. When $X_v(n) \gg c$ (i.e., the local memory intensity far exceeds the threshold), the exponent is a large positive number, so $e^{k(X_v(n)-c)} \rightarrow \infty$, and

$$W_{\text{rep}}(\mathbf{v}; n) \rightarrow 0.$$

The candidate site is strongly repelled, and the walker is very unlikely to move there.

3. When $X_v(n) = c$, the exponent is zero, so $e^0 = 1$, and

$$W_{\text{rep}}(v; n) = \frac{1}{2}.$$

Thus the threshold c is the intensity level at which the candidate site has exactly equal probability weight of being selected or rejected.

The steepness parameter k controls how rapidly the weight transitions from 1 to 0 around the threshold. To see this, we compute the derivative of W_{rep} with respect to X at the threshold point $X = c$:

$$\begin{aligned} \frac{\partial W_{\text{rep}}}{\partial X} &= -k \cdot \frac{e^{k(X_v(n)-c)}}{(1 + e^{k(X_v(n)-c)})^2} \\ \frac{\partial W_{\text{rep}}}{\partial X} &= -k \cdot W_{\text{rep}} \cdot (1 - W_{\text{rep}}) \\ \left. \frac{\partial W_{\text{rep}}}{\partial X} \right|_{X=c} &= -\frac{k}{4}. \end{aligned}$$

By deriving the logistic function, we found that its derivative is negative, which mean it is monotonic decreasing. The absolute value of the derivative is minimized when $W_{\text{rep}} = 0.5$, or when $X = c$. When $X = c$, the derivative $= -\frac{k}{4}$.

As k increases, the absolute value of slope gradient decrease. Thus larger values of k produce a sharper transition, while smaller values produce a more gradual response. In the limit $k \rightarrow \infty$, the Logistic function approaches a step function, revealing a hard threshold. In the limit $k \rightarrow 0$, the weight becomes constant $W_{\text{rep}} = 1/(1 + e^{-c})$ for all X , eliminating the repulsive mechanism entirely.

In Figure 4, assuming $c = 4.5$, six graphs of logistic function with different k are presented. When $k = 0$, there is no difference between the repulsive strength; when $k \rightarrow \infty$, any X above 4.5 result in a zero weight.

The Logistic function offers several advantages over alternative forms. First, it provides a clear threshold parameter c that separates 'acceptable' from 'unacceptable' memory intensities. Second, the smoothness of the function ensures that the transition probabilities vary continuously with the memory intensity. Third, the bounded range $W_{\text{rep}} \in (0, 1)$ ensures that no candidate site is ever completely forbidden.

In our simulations, we fix $k = 1.5$ as the baseline steepness parameter. This value was chosen to provide a sufficiently sharp transition while maintaining enough smoothness.

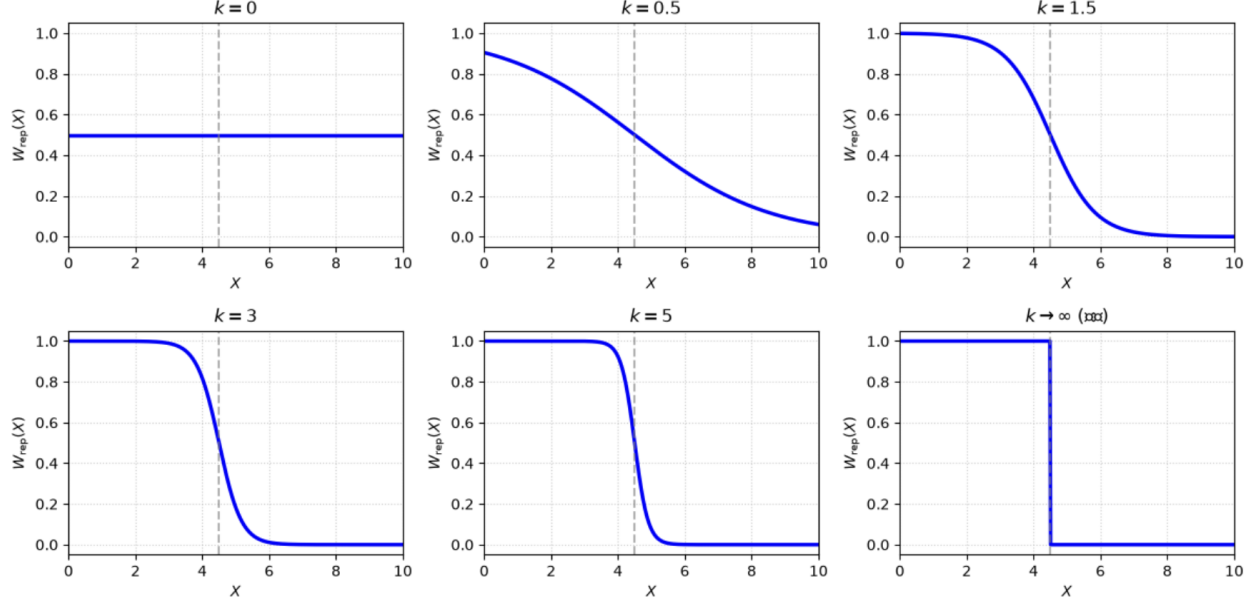


Figure 4: Logistic function with different k from zero to infinity

4. Central Attractive Force

The third novel component of our model is the central attractive force, which models the walker's tendency to return to a safe location. This mechanism is motivated by the observation that in many biological and human exploration scenarios, the walker experiences a competing force.

We define the center of the lattice as

$$O = \left(\frac{L}{2}, \frac{L}{2} \right).$$

For any site $v \in \Lambda$, let $d(v) = \|v - O\|_2$ denote the Euclidean distance from v to the center.

We define the attractive weight for candidate v as

$$A(v) = \frac{2}{1 + e^{\alpha \Delta(v)}}, \quad (5)$$

where $\alpha \geq 0$ is the attraction strength parameter. This function has the following properties:

- When $\Delta(v) = -1$ (step toward center), we have

$$A(v) = \frac{2}{1 + e^{-\alpha}} > 1,$$

so the candidate weight is amplified.

- When $\Delta(v) = +1$ (step away from center), we have

$$A(v) = \frac{2}{1 + e^{\alpha}} < 1,$$

so the candidate weight is suppressed.

- When $\alpha = 0$, we have $A(v) = 1$ for both cases.
- As $\alpha \rightarrow \infty$, $A(v) \rightarrow 2$ for inward steps and $A(v) \rightarrow 0$ for outward steps, setting boundaries for the attractive weight.

This formulation ensures that the attractive force is always present, regardless of the walker’s distance from the origin. A walker far from the center experiences the same relative amplification for inward steps as a walker near the center.

5. Composite Transition Probability

The composite weight for moving to candidate v is the product of the repulsive and attractive contributions:

$$W(v) = W_{\text{rep}}(v) \cdot A(v), \quad (6)$$

where $W_{\text{rep}}(v) = [1 + e^{k(X(v)-c)}]^{-1}$ as defined in Section 3.3, and $A(v)$ is given by Equation (3.4).

The transition probability is then obtained by normalizing over the four candidate directions:

$$\mathbb{P}(x_{n+1} = v \mid x_n, \mathcal{F}_n) = \frac{W(v)}{\sum_{h \in \mathcal{D}} W(x_n + h)}, \quad (7)$$

where $\mathcal{D}$ denotes the set of four cardinal directions, and $\mathcal{F}_n$ is the history of the walk up to time n .

Empirical analysis

1. Baseline Performance on the Finite Lattice

We first evaluate the model’s performance on the 100×100 lattice with no central attraction ($\eta = 0$). The baseline parameters are fixed at $k = 1.5$, $c = 5.5$, and $\lambda = 0.04$. We run $N = 10,000$ independent simulations, each with a maximum of 5×10^5 steps. The walker is considered to have *escaped* when it steps outside the lattice boundary.

Table 1 summarizes the escape-time distribution. Several features are noteworthy.

First, the escape probability is 100%: all 10,000 trials terminated by escape before reaching the step limit. This indicates that the repulsive mechanism is sufficiently strong to drive the walker outward with certainty.

Second, the mean escape time is $\bar{\tau} = 722.69$ steps, with a standard deviation of 500.20. This is substantially lower than the classical simple random walk with approximately 2500 steps. The reduction suggests that the memory-driven repulsion significantly accelerates exploration.

Third, the distribution is markedly right-skewed (skewness 1.96) and heavy-tailed (excess kurtosis 5.88). The median escape time (579 steps) is considerably lower than the mean, confirming that the distribution has a long right tail.

The quartiles reveal that 50% of the walks escape within 373 to 920 steps (interquartile range 547.25), indicating relatively consistent escape behavior, while a small fraction experience prolonged deviation.

Table 1: Escape-time statistics for the baseline model ($k = 1.5$, $c = 5.5$, $\lambda = 0.04$, $N = 10,000$).

Statistic	Value
Number of trials	10,000
Escape probability	100.0%
Mean	722.69
Median	579.00
Variance	250,202.82
Standard deviation	500.20
Standard error	5.00
95% CI	[712.88, 732.49]
Minimum	126
Maximum	4,886
Range	4,760
IQR	547.25
Skewness	1.9636
Excess kurtosis	5.8800
Percentiles	
1%	170
5%	222
10%	263
25% (Q1)	373
50% (Median)	579
75% (Q3)	920
90%	1,372
95%	1,709
99%	2,520

2. Parameter Sensitivity and Empirical Scaling Laws

To understand how the model parameters control escape time, we vary each of the three key parameters— k (Logistic steepness), c (repulsion threshold), and λ (memory decay rate)—while keeping the other two fixed at their baseline values. For each parameter value, we run 1,000 independent simulations and record the mean escape time.

For k , the escape time decreases exponentially with increasing steepness, following

$$\tau(k) = 1608.628 e^{-2.4155 k} + 666.870,$$

with $R^2 = 0.9956$. The extremely high R^2 value indicates that the escape time is almost perfectly described by an exponential decay in k .

For the repulsion threshold c , the escape time grows exponentially:

$$\tau(c) = 138.880 e^{0.3002 c} + 13.487,$$

with $R^2 = 0.9930$. This result is intuitive: a higher threshold means that the walker tolerates higher local memory intensity before repulsion becomes significant. The fitted constant term 13.487 is negligible compared to the exponential term over the scanned range.

For the memory decay rate λ , the escape time also increases exponentially:

$$\tau(\lambda) = 659.880 e^{8.7348 \lambda} - 193.017,$$

with $R^2 = 0.9877$. The negative constant term does not imply negative escape times for physically meaningful λ .

These empirical scaling laws provide a practical tool for parameter selection. Since k and λ act in opposite directions, and c is in the same direction as λ , there exists a trade-off space.

To quantify the distance at which our model effectively degenerates into the classical case, we computed the parameter values required to match the classical escape time while holding the other two parameters fixed at their baseline values. The results are striking: we require $k \approx -0.15$, $c \approx 10.2$, or $\lambda \approx 0.18$.

The negative steepness parameter ($k = -0.15$) have significant interpretation. When $k > 0$, higher local memory intensity $X(v)$ yields lower transition weight. A negative k inverts this relationship entirely: it causes the walker to be *attracted* to regions with high memory density, precisely the opposite of the repulsive mechanism of our model. The slightly negative value required to match the classical escape time suggests that to achieve classical behavior (3000 steps), the repulsive mechanism cannot exist.

The threshold value $c \approx 10.2$ is equally instructive. Within a 5×5 neighborhood, the total memory intensity $X(v)$ is the sum of up to 25 exponentially decaying terms. Setting $c = 10.2$ means that the repulsive weight remains close to 1 for the vast majority of moves, because $X(v)$ is almost always well below the threshold. This change means the walker no longer perceives any site as 'over-visited' and thus behaves more like a memoryless random walker.

The memory decay rate $\lambda \approx 0.18$ provides a third way to classical behavior. At this decay rate, the half-life of a memory trace is $t_{1/2} = \ln 2 / 0.18 \approx 3.85$ steps. After just 4 steps, a visit retains less than half its original intensity; after 10 steps, its contribution is $e^{-1.8} \approx 0.165$, already negligible. The walker's memory is so short that the memory mechanism becomes negligible, as well as the repulsive effect.

3. Numerical Evidence for Transience via Origin Returns

A direct analysis for recurrence versus transience in a random walk is the frequency of returns to the origin.

To assess this, we simulate the pure repulsive model ($\eta = 0$) on an infinite plane for $N = 500,000$ steps and record each return to the origin. The results are striking: over the entire simulation, the walker returns to the origin only a handful of times, and nearly all returns occur within the first several steps. Returns after several thousand steps are exceedingly rare.

This contrasts sharply with the classical simple random walk, which under identical conditions typically returns to the origin 5 to 20 times over 500,000 steps, with return times

distributed relatively uniformly across the simulation horizon. The classical walker, being recurrent, revisits the origin frequently and at arbitrarily large times.

The walker, once it has moved sufficiently far from the origin, rarely returns. While a rigorous proof of transience would require an analytic argument beyond the scope of this work and my capability, the numerical evidence is compelling and consistent across multiple independent simulations.

4. The Exploration-Return Phase Transition under Central Attraction

We now introduce the central attractive mechanism defined in Section 3.5, parameterized by $\alpha \geq 0$, which biases the walker toward steps that reduce its distance to the origin. At $\alpha = 0$, the model reduces to the pure repulsive walk studied above, which is numerically transient. As α increases, the attractive force competes with repulsion, and we observe a sharp transition in the walker’s escape behavior.

We fix the repulsive parameters at their baseline values ($k = 1.5$, $c = 5.5$, $\lambda = 0.04$) and vary the attraction strength α on the 100×100 lattice. For each α , we compute the mean escape time over 1,000 independent simulations. The results are summarized in Table 2.

At $\alpha = 0.2$, the mean escape time is approximately 2,700 steps, which is close to the classical random walk value of approximately 3000 steps. This suggests that the attractive force at this level is just strong enough to counteract the repulsive drive.

For $\alpha \geq 0.3$, the escape time increases dramatically: at $\alpha = 0.3$, it reaches approximately 7,800 steps; at $\alpha = 0.4$, it exceeds 28,000 steps; and at $\alpha = 0.5$, it reaches 120,000 steps. Beyond $\alpha \approx 0.2$, the walker becomes increasingly confined near the origin, requiring ever longer excursions to occasionally reach the boundary. The growth is super-exponential in α , indicating a dramatic transition in the walker’s long-term behavior. Specific equation of escape time versus α is difficult to obtain due to extremely long expected simulation time.

Another real-life interpretation of the parameter α is the afraidness of the explorer. If the explorer is afraid to explore the wider world, then α would be higher, forcing he/she to stay at the origin. On the other hand, if the explorer is an expert and would be delightful to explore, then α approaches 0, meaning insignificant central attraction.

Table 2: Mean escape time as a function of attraction strength α on the 100×100 lattice. Fixed repulsive parameters: $k = 1.5$, $c = 5.5$, $\lambda = 0.04$.

α	Mean escape time (steps)	Interpretation
0.0	723	Transient (very fast escape)
0.1	$\approx 1,300$	Likely-transient
0.2	$\approx 2,700$	Near classical (crossover region)
0.3	$\approx 7,800$	Attraction-dominated
0.4	$\approx 28,000$	Strongly trapped
0.5	$\approx 120,000$	Almost fully confined

We emphasize that while we cannot claim a rigorous mathematical proof of recurrence for

$\alpha > \alpha_c$, the numerical evidence—extremely long escape times, confinement near the origin, and the qualitative change in structure—strongly supports the interpretation of a transition from transient to recurrent behavior.

Conclusion

In this paper, we have introduced and systematically investigated a non-Markovian random walk model on the two-dimensional square lattice, driven by three interacting mechanisms: an exponentially decaying memory field, a Logistic soft repulsion based on local memory intensity, and an optional central attraction. Through extensive Monte Carlo simulations and statistical analysis, we have demonstrated that these mechanisms fundamentally alter the walker’s long-term behavior relative to the classical Pólya paradigm.

Our primary finding is that the pure repulsive model (without attraction) is numerically transient. On a finite lattice, the walker escapes faster than the classical random walk, with a mean escape time of approximately 723 steps compared to roughly 3000 steps for the memoryless case. On the infinite plane, returns to the origin after the initial transient are exceedingly rare, in stark contrast to the frequent and uniformly distributed returns observed in the classical recurrent walk. This provides strong numerical evidence that the memory-repulsion coupling drives the system into a transient regime.

We have quantified the model’s sensitivity to its three core parameters—the Logistic steepness k , the repulsion threshold c , and the memory decay rate λ —and derived empirical exponential scaling laws for each. These fits enable predictive control over the walker’s escape efficiency and reveal that the model degenerates into classical behavior only under extreme parameter values: a negative steepness, an excessively high threshold, or an exceedingly fast decay rate.

The introduction of central attraction reveals a sharp phase transition. At attraction strengths $\alpha \lesssim 0.2$, the walker remains transient and escapes efficiently. Beyond this critical value, however, the escape time grows explosively. We interpret this as a transition from transient to recurrent behavior, driven by the competition between outward repulsion and inward bias.

Looking forward, a rigorous analytic proof of transience for the pure repulsive model remains an open challenge, requiring new techniques for non-Markovian processes. Extending the model to three dimensions or to continuous space could reveal whether the observed transience is dimension-dependent. Additionally, the phase transition induced by central attraction may have implications for optimal search strategies and animal foraging theory, where the trade-off between exploration and safety is a central concern. Our framework offers a tractable computational structure for studying these broader questions.

References

- [1] G. Pólya, “Über eine Aufgabe der Wahrscheinlichkeitsrechnung betreffend die Irrfahrt im Straßennetz,” *Math. Ann.*, vol. 84, no. 1-2, pp. 149–160, 1921.
- [2] N. Madras and G. Slade, *The Self-Avoiding Walk*. Boston, MA: Birkhäuser, 1996.
- [3] D. J. Amit, G. Parisi, and L. Peliti, “Asymptotic behavior of the ‘true’ self-avoiding walk,” *Phys. Rev. B*, vol. 27, no. 3, pp. 1635–1645, 1983.
- [4] P. J. Flory, *Principles of Polymer Chemistry*. Ithaca, NY: Cornell University Press, 1953.
- [5] B. Nienhuis, “Exact critical point and critical exponents of $O(n)$ models in two dimensions,” *Phys. Rev. Lett.*, vol. 49, no. 15, pp. 1062–1065, 1982.
- [6] P. G. Doyle and J. L. Snell, *Random Walks and Electric Networks*, Carus Mathematical Monographs, no. 22. Washington, D.C.: Mathematical Association of America, 1984.
- [7] G. M. Viswanathan, S. V. Buldyrev, S. Havlin, M. G. E. da Luz, E. P. Raposo, and H. E. Stanley, “Optimizing the success of random searches,” *Nature*, vol. 401, pp. 911–914, 1999.

Appendix

```
import random
import math
import numpy as np

GRID_SIZE = 100
START_POS = (GRID_SIZE // 2, GRID_SIZE // 2)
NUM_REPEATS = 10000
MAX_STEPS = 500000

K = 1.5
C = 5.5
LAMBDA = 0.04

DIRS = [(0, 1), (1, 0), (0, -1), (-1, 0)]

NEIGHBORHOOD_OFFSETS = []
for dx in range(-2, 3):
    for dy in range(-2, 3):
        NEIGHBORHOOD_OFFSETS.append((dx, dy))

def compute_memory_strength(pos, last_seen, current_time):
    cx, cy = pos
    total_strength = 0.0
    for dx, dy in NEIGHBORHOOD_OFFSETS:
        nx, ny = cx + dx, cy + dy
        if 0 <= nx < GRID_SIZE and 0 <= ny < GRID_SIZE:
            last_t = last_seen[nx][ny]
            if last_t >= 0:
                total_strength += math.exp(-LAMBDA * (current_time - last_t))
    return total_strength

def run_single_walk():
    x, y = START_POS
    last_seen = [[-1] * GRID_SIZE for _ in range(GRID_SIZE)]
    last_seen[x][y] = 0
    steps = 0
    while steps < MAX_STEPS:
        options = []
        for dx, dy in DIRS:
            nx, ny = x + dx, y + dy
            inside = (0 <= nx < GRID_SIZE and 0 <= ny < GRID_SIZE)
            if inside:
                X = compute_memory_strength((nx, ny), last_seen, steps)
```

```

        weight = 1.0 / (1.0 + math.exp(K * (X - C)))
        options.append((nx, ny), weight, False))
    else:
        weight = 1.0 / (1.0 + math.exp(K * (0 - C)))
        options.append((nx, ny), weight, True))
    total_weight = sum(w for _, w, _ in options)
    r = random.random() * total_weight
    cumsum = 0.0
    chosen_idx = 0
    for i, (_, w, _) in enumerate(options):
        cumsum += w
        if r <= cumsum:
            chosen_idx = i
            break
    chosen_pos, _, is_escape = options[chosen_idx]
    if is_escape:
        return steps + 1, True
    x, y = chosen_pos
    steps += 1
    last_seen[x][y] = steps
    return MAX_STEPS, False

def calculate_statistics(data):
    if len(data) == 0:
        return None
    arr = np.array(data)
    n = len(arr)
    mean = np.mean(arr)
    median = np.median(arr)
    variance = np.var(arr, ddof=1)
    std_dev = np.std(arr, ddof=1)
    std_error = std_dev / np.sqrt(n)
    z_score = 1.96
    ci_lower = mean - z_score * std_error
    ci_upper = mean + z_score * std_error
    min_val = np.min(arr)
    max_val = np.max(arr)
    percentiles = {
        '1%': np.percentile(arr, 1),
        '5%': np.percentile(arr, 5),
        '10%': np.percentile(arr, 10),
        '25% (Q1)': np.percentile(arr, 25),
        '50% (Median)': np.percentile(arr, 50),
        '75% (Q3)': np.percentile(arr, 75),
        '90%': np.percentile(arr, 90),
    }

```

```

        '95%': np.percentile(arr, 95),
        '99%': np.percentile(arr, 99),
    }
    iqr = percentiles['75% (Q3)'] - percentiles['25% (Q1)']
    try:
        from scipy import stats
        skewness = stats.skew(arr)
        kurtosis = stats.kurtosis(arr)
    except (ImportError, NameError):
        m2 = np.mean((arr - mean)**2)
        m3 = np.mean((arr - mean)**3)
        m4 = np.mean((arr - mean)**4)
        skewness = m3 / (m2 ** 1.5) if m2 > 0 else 0
        kurtosis = m4 / (m2 ** 2) - 3
    return {
        'n': n, 'mean': mean, 'median': median, 'variance': variance,
        'std_dev': std_dev, 'std_error': std_error,
        'ci_95_lower': ci_lower, 'ci_95_upper': ci_upper,
        'min': min_val, 'max': max_val, 'range': max_val - min_val,
        'iqr': iqr, 'percentiles': percentiles,
        'skewness': skewness, 'kurtosis': kurtosis,
    }

def print_statistics_table(stats_dict, label):
    if stats_dict is None:
        print(f" {label}: No data")
        return
    print(f"\n [{label}]")
    print(f"      N: {stats_dict['n']}")
    print(f"      Mean: {stats_dict['mean']:.2f}")
    print(f"      Median: {stats_dict['median']:.2f}")
    print(f"      Variance: {stats_dict['variance']:.2f}")
    print(f"      Std Dev: {stats_dict['std_dev']:.2f}")
    print(f"      Std Error: {stats_dict['std_error']:.2f}")
    print(f"      95% CI: [{stats_dict['ci_95_lower']:.2f}, {stats_dict['ci_95_upper']:.2f}]")
    print(f"      Min: {stats_dict['min']:.0f}")
    print(f"      Max: {stats_dict['max']:.0f}")
    print(f"      Range: {stats_dict['range']:.0f}")
    print(f"      IQR: {stats_dict['iqr']:.2f}")
    print(f"      Skewness: {stats_dict['skewness']:.4f}")
    print(f"      Kurtosis: {stats_dict['kurtosis']:.4f}")
    print(f"\n      Percentiles:")
    for p, val in stats_dict['percentiles'].items():
        print(f"          {p}>12}: {val:.2f}")

```

```

print(f"Running {NUM_REPEATS} simulations (exponential memory, lambda={LAMBDA})...")
print(f"Logistic parameters: k={K}, c={C}")
print(f"Check:  $\exp(-{LAMBDA}*15) = \{\text{math.exp}(-LAMBDA*15):.3f\}$  (> 0.5)\n")

results = []
for i in range(NUM_REPEATS):
    steps, escaped = run_single_walk()
    results.append((steps, escaped))
    if (i + 1) % 1000 == 0:
        print(f"  Completed {i+1}/{NUM_REPEATS} runs")

escaped_steps = [s for s, e in results if e is True]
failed_steps = [s for s, e in results if e is False]
failed_count = len(failed_steps)

print("\n" + "=" * 70)
print("SUMMARY STATISTICS (exponential memory)")
print("=" * 70)

print(f"\nTotal runs:           {NUM_REPEATS}")
print(f"Escaped:                   {len(escaped_steps)} ({len(escaped_steps)/NUM_REPEATS*100:.1f}%)")
print(f"Failed (step limit): {failed_count} ({failed_count/NUM_REPEATS*100:.1f}%)")

escaped_stats = calculate_statistics(escaped_steps)
print_statistics_table(escaped_stats, "Escaped steps")

if failed_count > 0:
    failed_stats = calculate_statistics(failed_steps)
    print_statistics_table(failed_stats, "Failed steps")

```